Lead Architect — Dual-Model Counterpart System Design

Project: **MCProcessor** · Workspace: `GODMODE/Codebase` · Generated: 2026-05-21 · Repo: `Jose-Pedro/MCProcessor`

User request — Lead Architect mandate

Act as the Lead Architect in a dual-model agentic framework. We are establishing a persistent "Agent Counterpart" system where you serve as the strategic architect and a future remote model (Qwen 3.6) will serve as the primary coding engine — making you the supervisor. Remember the concept is *not* fast AI; we are focused on doing things right, not doing things now. Any task that the local models can perform should be done by them and await the time needed, being your responsibility to train them and revise them to the best of the models' abilities, always focusing on doing the least interactions possible and being patient for replies and tunings.

Objective: design and implement a high-efficiency long-term memory and training architecture with the following requirements:

1. **Dual-Model Roles**
   - **Local Model (Ollama):** "Knowledge Vault" and "Context Manager." Stores all consolidated training, architectural decisions, and session history.
   - **Remote Model (Qwen 3.6):** "Execution Engine" for complex coding tasks, guided by context provided by the local model.
2. **Knowledge Persistence & Consolidation**
   - Every new session begins by loading the "Consolidated Past Knowledge."
   - "Best Version" logic: avoid recording redundant chat history; distill lessons learned and optimal code patterns into a structured knowledge base (e.g. `.augment/` or `AGENTS.md`).
   - 100% functional accuracy and architectural integrity; prioritize the "right solution" over user-pleasing.
3. **Model Investigation**
   - Evaluate currently installed local models (`acidtib/qwen2.5-coder-cline:7b` , `qwen2.5-coder:7b-instruct` , `nemotron-3-nano:4b` , `nomic-embed-text`) to determine which is most efficient for "Knowledge Extraction" and "Context Compression."
   - Propose a method for using the embedding model (`nomic-embed-text`) to facilitate fast retrieval of long-term memory.
4. **Environment Continuity**
   - Define the "Bootstrap" process for starting a session so that no context or training progress is ever lost, allowing the environment to evolve as a long-term autonomous counterpart.

0. Critical clarification first

"Qwen 3.6" — I cannot identify this version unambiguously. The Qwen family today has:

- Qwen 2.5 (the family we have local: `qwen2.5-coder:7b`)
- Qwen 3 base + Qwen 3 Coder (30B MoE, ~17 GB Q4 — does *not* fit your 6 GB VRAM)
- No publicly released "Qwen 3.6" that I can verify

Before designing the remote half I need to know:

1. **What model exactly** — a hosted endpoint (Alibaba DashScope? OpenRouter? Together?), a self-hosted deployment on a bigger machine, or something else?
2. **Access mechanism** — API key + OpenAI-compatible HTTP, SSH-tunneled Ollama, Cline's built-in providers, …?
3. **Cost model** — free local-network / self-hosted, or paid per-token? This drives how aggressive we should be in delegating to local first.

The local-side architecture below **does not depend on this answer** — it's the foundation that holds regardless.

Operating principle (encoded as routing law)

> *The Lead Architect and the Remote Execution Engine are the most expensive resources in the system. No task reaches them that the local ensemble can perform. If the local ensemble is uncertain, it produces a structured failure report — never a guess — and the task is routed one level up.*

This means slow first runs (local models think; distillation takes time) but compounding efficiency — every distilled fact reduces future re-work.

1. System overview

```
SUPERVISOR (Lead Architect) — strategy, audit, escalation
    │
    ├─ delegates routine tasks to LOCAL ENSEMBLE (Ollama)
    │     ├─ acidtib qwen-cline 7b  → Cline agentic tool-use
    │     ├─ qwen2.5-coder 7b inst  → extraction / distillation
    │     ├─ nemotron-3-nano 4b     → fast classify / chunk summary
    │     └─ nomic-embed-text       → embeddings for retrieval
    │
    └─ assigns heavy coding to REMOTE EXEC ENGINE (TBD — "Qwen 3.6")
```

2. Model role assignments (and how I arrived at them)

Capability needed	Assigned model	Evidence / reasoning	Confidence
Cline agentic tool-use	`acidtib/qwen2.5-coder-cline:7b`	Smoke-tested ✅; 16 K context; explicitly fine-tuned for Cline's tool-call JSON.	High
Knowledge extraction (source file → structured facts)	`qwen2.5-coder:7b-instruct-q4_K_M`	16 K context handles big CDS files; instruct-tuned for "extract X from Y" patterns; same base as the Cline tune so structurally consistent.	Medium-high — needs a targeted test on a real `processflow.cds`-sized input
Context compression (raw chat → distilled lesson)	**Two-stage:** `nemotron-3-nano:4b` for chunk-summary pass, then `qwen2.5-coder:7b-instruct` for cross-chunk merge	Nemotron is 1.6× faster (31 vs 19 tok/s) but only 4 K context — perfect for chunk-by-chunk first pass. Qwen 7B does the unifying second pass where reasoning matters.	Medium — needs validation
Classification (tag a snippet: decision / fact / pattern / anti-pattern)	`nemotron-3-nano:4b`	Cheap discrete-output task, exactly what a fast small model is for.	High
Embeddings	`nomic-embed-text`	Only embedding model on the box; 768-dim; 8192-token input; Ollama `/api/embeddings`.	High
Merge arbiter (two memory entries disagree)	Escalate to Lead Architect	Governance call, not a generation task.	High
Complex multi-file refactor	Remote Execution Engine	Above local capability ceiling.	Pending remote-model confirmation

Caveat: the "context compression" and "knowledge extraction" assignments are based on architectural intuition + the smoke-bench tok/s data. They have *not* yet been tested on a real distillation task against the repo. I will not commit to these assignments without running a 5-minute targeted test — see Phase 0 below.

3. Storage layout (proposed — no files created yet)

```
.augment/
├── AGENTS.md                       (existing — root behavioral contract)
├── SESSION_LOG.md                  (existing — rolling human-readable log)
├── memory/
│   ├── README.md                   (how the memory system works — for humans)
│   ├── journal.jsonl               (APPEND-ONLY: every fact ever recorded, with embedding
hash)
│   ├── consolidated.md             (BEST-VERSION view: deduplicated, human-readable)
│   ├── decisions.md                (architectural decision records — ADRs)
│   ├── patterns.md                 (reusable code patterns confirmed in repo)
│   └── anti-patterns.md            (things tried that failed — never re-try without cause)
├── embeddings/
│   └── store.sqlite                (SQLite: chunk_id, text, source_file, vector BLOB,
created_at)
├── prompts/
│   ├── distill.txt                 (raw chat → structured facts)
│   ├── merge.txt                   (existing entry + new info → best version)
│   ├── extract.txt                 (source file → structured facts)
│   └── classify.txt                (snippet → category)
└── scripts/
    ├── ollama-server.ps1           (existing-ish: ensure Ollama is up)
    ├── memory-write.ps1            (NEW: append fact + embed + merge into consolidated)
    ├── memory-query.ps1            (NEW: semantic search → top-K chunks)
    ├── session-bootstrap.ps1       (NEW: load context at session start)
    ├── session-checkpoint.ps1      (NEW: distill session → memory)
    └── router.ps1                  (NEW: pick the right local model for a task)
```

Two important design choices:

1. **`journal.jsonl` is append-only and immutable.** Every fact ever extracted, with timestamp + embedding hash + source. This is the audit trail. If the consolidated view ever rots, we re-derive it from the journal.
2. **`consolidated.md` is regenerable.** It's the "best version" — but it's never the source of truth. The journal is.

This separation means: **we can never lose knowledge.** Even if a distillation pass introduces a bad merge, rolling back is "regenerate `consolidated.md` from `journal.jsonl`."

4. The "Best Version" merge protocol

When `memory-write.ps1` receives a new fact:

```
1. classify(new_fact)              → category   [nemotron-3-nano]
2. embed(new_fact)                 → vector     [nomic-embed-text]
3. append to journal.jsonl         (immutable)
4. top-3 = query(vector, threshold=0.85)
5. IF top-3 empty:
       → new entry in consolidated.md
6. ELSE for each near-neighbor:
       → run merge.txt prompt:
           "Given EXISTING entry [X] and NEW fact [Y], produce the unified
            best version. Rules:
              - Do NOT add facts not present in either source.
              - Prefer specific over vague (file:line refs win over 'somewhere').
              - If contradicted, mark with !CONTRADICTION! and stop.
              - Output ONLY the merged entry, no preamble."
       → [qwen2.5-coder 7b instruct]
7. IF !CONTRADICTION! in output:
       → escalate to Lead Architect — do not auto-resolve
8. ELSE replace the existing entry with the merged version.
9. Re-embed merged entry, update embeddings store.
```

Guardrails preventing hallucination:

- Merge prompt explicitly forbids adding new facts.
- The journal still has both originals for rollback.
- A contradiction stops the pipeline rather than being silently picked.

5. Retrieval (RAG) design

Why SQLite, not a vector DB: for a corpus under ~10 K chunks (we'll be far below that for years), an in-process SQLite with cosine-similarity in PowerShell beats any heavyweight vector DB on operational simplicity. Zero servers, zero ports, zero install. If we ever exceed 10 K we migrate to `sqlite-vss` or `pgvector`.

Storage schema:

```
CREATE TABLE chunks (
  id          INTEGER PRIMARY KEY,
  category    TEXT NOT NULL,         -- decision|fact|pattern|anti-pattern|lesson
  source      TEXT,                  -- e.g. 'session:2026-05-21:checkpoint-3'
  text        TEXT NOT NULL,
  vector      BLOB NOT NULL,         -- 768 * 4 bytes (float32)
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  superseded_by INTEGER REFERENCES chunks(id)  -- soft-delete after merge
);
CREATE INDEX idx_category ON chunks(category) WHERE superseded_by IS NULL;
```

Query path:

```
query("How does prv-bts processflow relate to MCProcessor?")
  → embed via nomic-embed-text          (~50 ms)
  → SELECT id, text, dot_product(vector, :q) AS score
       FROM chunks WHERE superseded_by IS NULL
    ORDER BY score DESC LIMIT 10        (~5-20 ms for <10K rows)
  → filter by score > 0.55              (configurable threshold)
  → return top-K with provenance
```

Cosine similarity over 768-dim float32 vectors on small N can be done in pure PowerShell in ~10 ms; if it gets slow we move it to a Python helper.

6. Session bootstrap protocol

When a new Augment session opens:

1. **Existing** (per `AGENTS.md`): read `AGENTS.md`, read `SESSION_LOG.md` tail.
2. **NEW step:** Lead Architect calls `session-bootstrap.ps1` with the user's first prompt as input:

   ```
   user_prompt → embed → top-K memory chunks → render preamble
   ```

3. Preamble format (~30–50 lines max):

   ```
   ## Loaded from long-term memory (relevance to current prompt)

   [0.91] DECISION: prv-bts-nodejs not imported; processflow consumed via …
          source: session 2026-05-21 checkpoint-2
   [0.87] FACT: SCREEN_CONTROLS table lives in cmm-ddl-hana/db/src/cmm/…
          source: codebase scan 2026-05-21
   [0.83] PATTERN: every CAP project binds same HANA tables via …
   …
   ```

4. Lead Architect acknowledges loaded context in 1–2 lines to the user, then proceeds.

Crucially: if the user's first prompt is `checkpoint` / `save progress`, bootstrap is skipped and we jump to the checkpoint protocol instead.

7. Session checkpoint protocol

On user "checkpoint" / "save progress":

1. Lead Architect collects the session's salient turns (no raw chat dump).
2. Local pipeline:
   - **nemotron-3-nano** does chunk-by-chunk classification (decision / fact / pattern / lesson / noise)
   - **qwen2.5-coder 7b instruct** does the structured extraction prompt per non-noise chunk
3. For each extracted fact: pipe through `memory-write.ps1` (which handles the merge logic from §4).
4. Update `SESSION_LOG.md` per existing AGENTS.md template — but the entry references *which memory IDs were added/merged*, not the raw content. Memory is the source of truth; SESSION_LOG is the

human-readable timeline.

Result: every checkpoint **grows the consolidated knowledge base without growing the chat-replay log.**

8. Phased build plan

Explicit go/no-go between phases. No phase exceeds ~30 minutes wall-clock of build time.

Phase 0 — Targeted local-model validation (~10 min, prerequisite)

Before committing the role assignments in §2, test them:

- Feed `Prv\prv-bts-nodejs\db\processflow.cds` (~7 KB) to `qwen2.5-coder 7b instruct` with the extraction prompt → measure: (a) does it produce structured output? (b) does it hallucinate fields?
- Feed a 2-KB chat snippet to `nemotron-3-nano` with classification prompt → measure correctness.
- Feed an embedding query to `nomic-embed-text` → measure round-trip latency.
- **Output:** a one-page validation report; if any model fails its assigned role, reassign before building.

Phase 1 — Memory primitives (~30 min build)

- Create `.augment/memory/`, `.augment/embeddings/`, `.augment/prompts/`, `.augment/scripts/`
- Write `memory-write.ps1`, `memory-query.ps1` (with SQLite schema)
- Write the three prompt files (distill, merge, classify)
- Seed `journal.jsonl` and `consolidated.md` with the **already-captured** decisions from this and the prior session (MCProcessor scope, no prv-bts import, model lineup, the 5 architectural decisions, etc.) — this becomes the bootstrap data, NOT new content
- Smoke-test: write a fact, query for it, confirm round-trip

Phase 2 — Bootstrap + checkpoint scripts (~20 min build)

- Write `session-bootstrap.ps1` and `session-checkpoint.ps1`
- Wire them into `AGENTS.md` so future Augment sessions invoke them
- Smoke-test by simulating a session bootstrap

Phase 3 — Remote-engine integration (blocked on Qwen 3.6 clarification)

- Confirm remote model identity + access mechanism
- Define routing rules: "when does Lead Architect escalate from local to remote?"
- Wire the remote engine (likely via Cline's provider config or a thin CLI shim)
- Test escalation end-to-end on a real coding task

Phase 4 — Self-evolution loop (later)

- Periodic "memory health check": detect stale entries, contradiction clusters, low-confidence facts
- Lead Architect reviews, accepts/rejects pruning suggestions

9. What I need from you before I build anything

1. **Confirm "Qwen 3.6" identity** — model, host, access. (Blocks Phase 3 only; Phases 0–2 proceed regardless.)
2. **Confirm storage location** — `.augment/` under the workspace root, as proposed? Or a separate folder so memory persists across workspaces?
3. **Confirm Phase 0 (validation tests) goes first** — small, but ~10 min of local-model time and produces hard evidence for or against the proposed role assignments. Alternative: skip and trust intuition, build Phase 1 directly, fix roles later if wrong.
4. **Pending decisions from the previous design pass** (still open, mostly orthogonal — but 4d affects script paths):
   - 4a. Delete empty `cmm-processflow-*` placeholders?
   - 4b. Fix `prv-screeencontrol-ui5` git merge markers?
   - 4c. Have local model digest `cmm-sfr-interface-nodejs` design docs (perfect Phase 0+ workload)?
   - 4d. Rename `prv-intprojects-*` → `prv-mcprocessor-*`?

Recommendation

- **Now:** answer #1 (Qwen 3.6 identity) and #3 (yes/no to Phase 0 validation). Phase 0 is the cheapest possible reality-check before locking the architecture.
- **Then:** run Phase 0, report back validation results, propose any role adjustments. If validation passes, you say `proceed` and Phase 1 is built.
- **Optionally in parallel:** if you give #4c (digest the SFR docs), that becomes the *test workload* for the Phase 0 distillation pass — we learn something useful about prior-art while validating the model assignments. Two birds, one local model.

End of document · Generated by Lead Architect (Augment Agent / Claude Opus 4.7) · Source: `.augment/exports/lead-architect-design.html`